

Kairox Leather Intelligence Platform

Backend API Reference — Frontend Integration Guide

Modules covered: **Imports, Clients, Production, Analytics.**

Every endpoint with its request, response, and purpose, plus the end-to-end product flow.

Base URL: `/api/v1` · Auth: JWT Bearer · Version reflects per-piece tracking + SKU codes.

1 · Conventions

Base URL & versioning

Every route below is mounted under `/api/v1`. A path shown as `/production/scan` is called at `/api/v1/production/scan`.

Authentication

All endpoints require a JWT obtained from `POST /api/v1/auth/login`. Send it on every request as `Authorization: Bearer <token>`. Some routes additionally require a role — noted per endpoint. Roles: **DIRECT_MANAGER** (full access), **HR**, **EMPLOYEE** (shop floor), **CLIENT** (sees only their own orders).

Identifiers

Internal keys are UUIDs (`sku_id`, `style_id`, `order_id`). The UI should never show a UUID — every list endpoint returns a human label or code (`sku.code`, `order_number`, piece code) for display and selection. Production accepts `sku_code` in place of `sku_id` so the UI passes the readable code.

Errors

Standard FastAPI shape. A failed request returns an HTTP status (400 validation, 401 no/expired token, 403 wrong role, 404 not found) with a JSON body:

```
{ "detail": "Employee is not checked in today – mark attendance first." }
```

Dates are ISO YYYY-MM-DD; timestamps are ISO-8601. Quantities are integers.

2 · Product Flow

The platform digitises one make-to-order leather garment factory, from a buyer's breakdown sheet down to tracking every physical piece through every production stage. The frontend drives this lifecycle:

1. Ingest the breakdown sheet

A manager uploads the buyer's Excel breakdown (e.g. johnpeter.xlsx) via `POST /imports/preview` (dry-run — shows what will be created, no writes), reviews the summary + warnings, then commits with `POST /imports/commit`. This creates the Client, ClientOrder, Styles, and one SKU per (style · colour · size) — each SKU getting a readable **code** like JP-CLERMONT-DARK_BROWN-46. Per-date order lines are preserved.

2. Browse orders & SKUs

The UI lists clients (`GET /clients`), their orders (`GET /clients/{id}/orders`), and the pickable SKUs (`GET /production/skus`) which come back with a friendly label and code — never a raw UUID.

3. Cut (mint pieces)

At the cutting table the manager selects a SKU and a quantity and calls `POST /production/cutting`. The backend mints N physical pieces, each numbered 1..N within the SKU and each carrying a unique printed code (JP-CLERMONT-DARK_BROWN-46-005). Those codes go on the traveler cards.

4. Scan each stage

At every later stage (Fusing, Pasting, Shell, L/A, Lining Stitch, FF) the manager selects the SKU + operation + worker and types the piece numbers handled that day via `POST /production/scan`. Unknown numbers are reported, not written; a piece logged again at a stage it already passed is flagged as **rework**, never rejected.

5. Track & report

The analytics drill-down mirrors the hierarchy: order → styles (`/analytics/orders/{id}/tree`) → a style's pieces + stage history (`/analytics/styles/{id}/detail`) → one piece's full traveler (`/analytics/pieces/detail`). Dashboards add factory totals, bottleneck alerts, and sea-freight risk.

Data hierarchy

```
Client
  ■■ ClientOrder      (order_number, ship_mode, sea_cutoff_date)
    ■■ Style          (name, article)
      ■■ SKU          (code, colour, size, qty_ordered)
        ■■ SkuOrderLine (order_date, qty)      # per-date breakdown
          ■■ Piece      (code, seq 1..N)        # one physical garment
            ■■ ProductionEvent (operation, employee, work_date) # one scan
```

3 · Imports

Turns an uploaded Excel breakdown sheet into clients, orders, styles and SKUs. Two-step: preview (safe dry-run) then commit (writes, idempotent — re-uploading the same file replaces rather than duplicates).

POST `/imports/preview`

Dry-run: parse the uploaded workbook and return a structured summary with warnings. Nothing is written. Use this to let the manager confirm before commit.

Auth: DIRECT_MANAGER

PARAMETERS

file multipart/form-data — the .xlsx / .xlsm workbook

REQUEST

```
Content-Type: multipart/form-data

file=<johnpeter.xlsx>
```

RESPONSE

```
{
  "clients": {
    "JP": {
      "order_lines": 68,
      "pieces_ordered": 1234,
      "styles": ["ADELE KNIT", "CARNABY", "CLERMONT", "CLERMONT + VEST"],
      "production_cards": 0,
      "warnings": []
    }
  },
  "sheets": [ { "sheet": "Sheet1", "type": "order", "rows": 70 } ],
  "total_warnings": 0
}
```

- Grouped by client key (derived from the sheet). `pieces_ordered` is the summed quantity.
- Show warnings prominently — they flag rows the parser was unsure about.

POST **/imports/commit**

Parse, validate, and WRITE the workbook to the database. Idempotent: a client's prior styles/SKUs are cleared first, so re-committing the same sheet yields the same totals. Returns the preview summary plus what was written.

Auth: DIRECT_MANAGER

PARAMETERS

file multipart/form-data — the same .xlsx workbook

REQUEST

```
Content-Type: multipart/form-data

file=<johnpeter.xlsx>
```

RESPONSE

```
{
  "summary": { ...same shape as /imports/preview... },
  "written": {
    "clients": 1, "styles": 17,
    "skus_created": 120, "skus_updated": 0,
    "operations": 0, "rates": 0
  }
}
```

- After a successful commit, refresh the client/order/SKU lists.
- Each created SKU now carries a generated code; per-date rows are stored as order lines.

4 • Clients

Read clients and their orders. Orders nest their styles, and styles nest their SKUs — one call gives the whole order tree for a client.

GET **/clients**

List all clients.

Auth: Any authenticated user

RESPONSE

```
[
  {
    "id": "8f3c...uuid", "name": "John Peter", "country": "IT",
    "code": "JP", "currency": "EUR", "default_size_system": "EU"
  }
]
```

POST /clients

Create a client.

Auth: DIRECT_MANAGER

REQUEST

```
{ "name": "John Peter", "country": "IT" }
```

RESPONSE

```
{
  "id": "8f3c...uuid", "name": "John Peter", "country": "IT",
  "code": null, "currency": null, "default_size_system": null
}
```

GET /clients/{client_id}/orders

All orders for a client, each with its styles and their SKUs nested inside. A CLIENT-role user may only read their own `client_id`.

Auth: Any authenticated user (CLIENT restricted to self)

PARAMETERS

`client_id` UUID (path) — the client

RESPONSE

```
[
  {
    "id": "d21a...uuid", "order_number": "JP-P0",
    "order_date": "2025-02-23", "delivery_deadline": null,
    "sea_cutoff_date": "2025-04-10", "ship_mode": "SEA",
    "currency": "EUR", "agent": null, "line": null,
    "styles": [
      {
        "id": "aa11...uuid", "name": "CLERMONT", "article": "GOAT SUEDE",
        "gender": null, "unit_price": null, "currency": null,
        "skus": [
          { "id": "bb22...uuid", "color_code": "DARK BROWN",
            "color_name": "DARK BROWN", "size": "46", "qty_ordered": 24 }
        ]
      }
    ]
  }
]
```

- This is the richest read in the clients module — use it to render the full order → style → SKU tree without extra calls.

5 • Production

The shop-floor core: list operations, pick SKUs, cut pieces, and scan them through each stage. A **piece** is one physical garment; its identity is (SKU, seq) and it carries a printed code. Scanning is permissive — rework loops are recorded, not blocked.

GET**/production/operations**

List the production stages in order (Cutting → FF).

Auth: Any authenticated user

RESPONSE

```
[
  { "id": "op1...uuid", "code": "CUTTING", "label": "Cutting", "sequence": 1 },
  { "id": "op2...uuid", "code": "FUSING", "label": "Fusing", "sequence": 2 }
]
```

GET**/production/skus**

Friendly SKU picker for the log screens. Returns each SKU's code and a readable label so the user selects by name, never by UUID. Filter by order or style.

Auth: Any authenticated user

PARAMETERS

order_id	UUID (query, optional) — limit to one order
style_id	UUID (query, optional) — limit to one style

RESPONSE

```
[
  {
    "sku_id": "bb22...uuid",
    "code": "JP-CLERMONT-DARK_BROWN-46",
    "label": "CLERMONT \u00b7 DARK BROWN \u00b7 46",
    "order_number": "JP-PO", "style_name": "CLERMONT",
    "color_code": "DARK BROWN", "color_name": "DARK BROWN",
    "size": "46", "qty_ordered": 24
  }
]
```

- Populate SKU dropdowns from here. Pass the chosen code to cutting/scan as sku_code.

POST**/production/cutting**

Mint N pieces for a SKU at the cutting stage. Creates N piece records (seq 1..N) and N cutting events, and returns the piece codes to print on traveler cards. Accepts sku_code (preferred) or sku_id.

Auth: Role permitted for CUTTING (DIRECT_MANAGER bypasses)

REQUEST

```
{
  "sku_code": "JP-CLERMONT-DARK_BROWN-46",
  "employee_id": "e1...uuid",
  "work_date": "2026-07-13",
  "count": 21
}
```

RESPONSE

```
{
  "sku_id": "bb22...uuid", "count": 21,
  "pieces": [
    { "id": "p1...uuid", "code": "JP-CLERMONT-DARK_BROWN-46-001",
      "seq": 1, "sku_id": "bb22...uuid", "current_operation_id": "op1...uuid" }
  ]
}
```

- seq continues from any pieces already cut for the SKU (cut in batches across days).
- The employee must be checked in today if work_date is today (else 400).

POST **/production/scan**

Log a batch of pieces at one operation. Primary path: pick the SKU and type the piece numbers present (sku_code/sku_id + piece_seqs). Fallback: pass full piece codes. Partial-accept — unknown items are reported, valid ones committed.

Auth: Role permitted for the operation (DIRECT_MANAGER bypasses)

REQUEST

```
// primary – SKU-scoped, type the numbers
{
  "operation_id": "op2...uuid",
  "employee_id": "e1...uuid",
  "work_date": "2026-07-13",
  "sku_code": "JP-CLERMONT-DARK_BROWN-46",
  "piece_seqs": [1, 2, 3, 5, 8]
}

// fallback – full codes (typed / future scan-gun)
{
  "operation_id": "op2...uuid", "employee_id": "e1...uuid",
  "work_date": "2026-07-13",
  "piece_codes": ["JP-CLERMONT-DARK_BROWN-46-005"]
}
```

RESPONSE

```
{
  "operation": "FUSING",
  "count_logged": 5,
  "logged": ["JP-CLERMONT-DARK_BROWN-46-001", "...-002"],
  "rework": ["JP-CLERMONT-DARK_BROWN-46-005"],
  "not_found": ["#99"]
}
```

- Provide EITHER (sku + piece_seqs) OR piece_codes, else 400.
- rework = pieces already seen at this stage (still logged). not_found = skipped, never written. Surface both back to the user.

GET **/production/styles/{style_id}/progress**

Live stage totals for a style: how many pieces have passed each operation, against the ordered quantity — the 'Carnaby card' view.

Auth: Any authenticated user

PARAMETERS

style_id UUID (path)

RESPONSE

```
{
  "style_id": "aa11...uuid", "style_name": "CLERMONT", "qty_ordered": 152,
  "stages": { "CUTTING": 152, "FUSING": 152, "PASTING": 150, "FF": 120 }
}
```

POST **/production/events**

Legacy quantity-based entry (no piece linkage). Kept for back-compat; prefer /cutting and /scan for per-piece tracking.

Auth: Role permitted for the operation

REQUEST

```
{
  "sku_id": "bb22...uuid", "operation_id": "op2...uuid",
  "employee_id": "e1...uuid", "work_date": "2026-07-13", "qty": 10
}
```

RESPONSE

```
{
  "id": "ev...uuid", "sku_id": "bb22...uuid", "operation_id": "op2...uuid",
  "employee_id": "e1...uuid", "work_date": "2026-07-13", "qty": 10,
  "entered_by": "Rajesh (manager)", "piece_id": null
}
```

GET **/production/events**

List raw production events, filterable. Mostly for audit/debug; the analytics drill-down is the user-facing view.

Auth: Any authenticated user

PARAMETERS

sku_id	UUID (query, optional)
employee_id	UUID (query, optional)
start	date (query, optional)
end	date (query, optional)

RESPONSE

```
[ { "id": "ev...uuid", "sku_id": "bb22...uuid", "operation_id": "op1...uuid",
  "employee_id": "e1...uuid", "work_date": "2026-07-13", "qty": 1,
  "entered_by": "Rajesh", "piece_id": "p1...uuid" } ]
```

6 • Analytics

Read-only reporting. The three drill-down endpoints mirror the order → style → piece hierarchy, so the UI can expand progressively. Each stage row carries the employee and date/time; rework passes are flagged.

GET **/analytics/overview**

Factory-wide totals for the dashboard header.

Auth: Any authenticated user

RESPONSE

```
{
  "clients": 3, "styles": 17,
  "total_pieces_ordered": 4820, "total_operations_logged": 15330
}
```

GET

/analytics/orders/{order_id}/tree

LEVEL 1 — order landing view. The order plus its styles, each summarised by piece count and how those pieces are distributed across current stages. Use it to render the order's style list; expand a style via the next endpoint.

Auth: Any authenticated user

PARAMETERS

order_id UUID (path)

RESPONSE

```
{
  "order_id": "d21a...uuid", "order_number": "JP-P0", "client": "John Peter",
  "styles": [
    {
      "style_id": "aa11...uuid", "style_name": "CLERMONT", "article": "GOAT SUEDE",
      "piece_count": 60,
      "stage_counts": { "CUTTING": 20, "FUSING": 20, "FF": 12, "UNSTARTED": 8 }
    }
  ]
}
```

- UNSTARTED = minted pieces with no scan yet (current stage null).

GET

/analytics/styles/{style_id}/detail

LEVEL 2 — a single style with every one of its pieces, each carrying its full stage history. This is what to show when the user selects a style.

Auth: Any authenticated user

PARAMETERS

style_id UUID (path)

RESPONSE

```
{
  "style_id": "aa11...uuid", "style_name": "CLERMONT", "article": "GOAT SUEDE",
  "order_number": "JP-P0", "client": "John Peter",
  "pieces": [
    {
      "piece_id": "p5...uuid", "bundle_id": "JP-CLERMONT-DARK_BROWN-46-005",
      "seq": 5, "sku_code": "JP-CLERMONT-DARK_BROWN-46",
      "colour": "DARK BROWN", "size": "46", "current_stage": "PASTING",
      "stages": [
        { "stage_code": "CUTTING", "stage_label": "Cutting",
          "employee_name": "Anil", "work_date": "2026-07-13",
          "logged_at": "2026-07-13T09:12:04", "entered_by": "Rajesh",
          "is_rework": false }
      ]
    }
  ]
}
```

- current_stage is the latest event (a reworked piece reads back at the earlier stage).
- Each stage lists who did it and when; is_rework=true marks a repeat at that stage.

GET **/analytics/pieces/detail**

LEVEL 3 — one piece's full traveler. Look it up by piece_code, or by sku_code + seq. Returns the order/style/SKU header plus the ordered stage history.

Auth: Any authenticated user

PARAMETERS

piece_code	string (query) — e.g. JP-CLERMONT-DARK_BROWN-46-005
sku_code	string (query) — used with seq instead of piece_code
seq	int (query) — the piece number within the SKU

RESPONSE

```
{
  "bundle_id": "JP-CLERMONT-DARK_BROWN-46-005", "seq": 5,
  "sku_code": "JP-CLERMONT-DARK_BROWN-46",
  "sku_label": "CLERMONT \u00b7 DARK BROWN \u00b7 46",
  "colour": "DARK BROWN", "size": "46",
  "style_name": "CLERMONT", "article": "GOAT SUEDE",
  "order_number": "JP-P0", "client": "John Peter",
  "current_stage": "PASTING",
  "stages": [
    { "stage_code": "CUTTING", "stage_label": "Cutting", "employee_name": "Anil",
      "work_date": "2026-07-13", "logged_at": "2026-07-13T09:12:04",
      "entered_by": "Rajesh", "is_rework": false }
  ]
}
```

GET **/analytics/alerts/stage-spread**

Bottleneck alert: styles where a downstream stage lags cutting by >50% (true work-in-flight under per-piece tracking).

Auth: Any authenticated user

RESPONSE

```
[
  { "style": "CLERMONT", "stage": "FF", "cut": 152, "reached_stage": 120,
    "gap": 32, "severity": "medium" }
]
```

GET **/analytics/alerts/freight-risk**

Sea-freight risk: purchase orders nearing their sea cutoff while still in production (missing it forces costly air freight).

Auth: Any authenticated user

PARAMETERS

today	date (query, optional) — override 'today' for testing
-------	---

RESPONSE

```
[
  { "order_number": "JP-P0", "sea_cutoff": "2026-07-18", "days_left": 5,
    "ordered": 1234, "finished": 800, "pct_complete": 64.8, "risk": "high" }
]
```

Generated for the Kairox frontend team. Field names match the backend schemas exactly; example UUIDs/values are illustrative. Production and Analytics reflect the per-piece + SKU-code build.